

A Layered GPU Scheduling Architecture for Iterative Generative Workloads

Qinke Ni
225045036

The Chinese University of Hong Kong, Shenzhen
April 2026

Abstract

Modern generative AI workloads—including text-to-speech (TTS), diffusion models, and large language model (LLM) inference and training—pose severe challenges to GPU scheduling due to their iterative, latency-sensitive, and bursty nature. Existing GPU sharing mechanisms force operators to choose between high utilization and low tail latency: coarse spatial sharing raises throughput but creates interference, while temporal sharing protects priority but suffers from long preemption delays on closed-source GPUs. This report synthesizes three recent systems—KRYPTON, LithOS, and Hummingbird—into a unified layered architecture that decomposes the GPU scheduling problem into three coordinated layers: *entrance control* via kernel-space interception and isolated virtual GPU construction (KRYPTON), *execution abstraction* via TPC-level scheduling and kernel atomization (LithOS), and *emergency preemption* via microsecond-scale split-kernel scheduling and bubble harvesting (Hummingbird). We present the design rationale, methods, and experimental evidence for each layer, and argue that the three layers form a closed-loop system that simultaneously addresses deployability, efficiency, and strict tail-latency protection. Experimental results show that the layered approach reduces GPU fleet requirements by 32.1%, lowers tail latency by up to 13× compared to MPS, and achieves 9.7× higher SLO attainment than state-of-the-art spatial sharing—all without sacrificing best-effort throughput.

Keywords: GPU scheduling, iterative generative workloads, spatial-temporal sharing, SLO-oriented preemption, GPU virtualization

Contents

Abstract	i
List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Background and Motivation	1
1.2 The Layered Scheduling Problem	3
1.3 Contributions	5
1.4 Structure of the Report	6
2 Related Work	7
2.1 Temporal GPU Sharing	7
2.2 Spatial GPU Sharing	9
2.3 SLO-Oriented and Priority-Aware Scheduling	10
2.4 GPU Virtualization and API Remoting	12
2.5 GPU Operating System Abstractions	13
2.6 Positioning of KRYPTON, LithOS, and Hummingbird	14
3 Proposed Methods	17
3.1 KRYPTON: Control at the GPU Entrance	17

3.1.1	Design Goals and Motivating Observations	17
3.1.2	The IGPU Abstraction	18
3.1.3	Kernel-Space Interception	19
3.1.4	System Components and Operation	20
3.1.5	Layer Contribution	21
3.2	LithOS: Execution Abstraction Inside the GPU	23
3.2.1	Design Goals and Motivating Observations	23
3.2.2	GPU Hardware Background	23
3.2.3	TPC-Level Scheduling and TPC Stealing	24
3.2.4	Kernel Atomization	25
3.2.5	Hardware Right-Sizing	26
3.2.6	Transparent Power Management	26
3.2.7	Layer Contribution	27
3.3	Hummingbird: Emergency Preemption and Bubble Harvesting	27
3.3.1	Design Goals and Motivating Observations	27
3.3.2	Kernel Splitting	30
3.3.3	Bubble-Aware Runtime Scheduling	31
3.3.4	Hierarchical Memory Offloading	32
3.3.5	Layer Contribution	32
3.4	A Unified Layered Interpretation	35
3.4.1	How the Three Layers Compose	35
3.4.2	Inter-Layer Interactions and Design Tensions	36
3.4.3	Why All Three Layers Are Necessary	37
4	Experiments and Conclusion	38
4.1	Experimental Setup	38
4.2	KRYPTON: Compatibility, Isolation, and Cluster Efficiency . . .	40
4.2.1	GPU Fleet Reduction	40

4.2.2	Throughput Accuracy and QoS	40
4.2.3	Production Deployment	41
4.3	LithOS: Fine-Grained Scheduling and Energy Efficiency	43
4.3.1	Inference-Only Multitenancy	43
4.3.2	Hybrid Inference-Training Multitenancy	43
4.3.3	Resource Efficiency: Right-Sizing and DVFS	45
4.4	Hummingbird: SLO Protection and Bubble Harvesting	46
4.4.1	Single-GPU Priority Scheduling	46
4.4.2	Memory-Intensive Workloads	48
4.4.3	Multi-GPU Distributed Scheduling	48
4.4.4	Cross-GPU Generalization	51
4.4.5	Preemption Delay and Overhead	51
4.5	Comparative Analysis	51
4.6	Limitations and Future Directions	55
4.7	Conclusion	56

Bibliography	59
---------------------	-----------

List of Figures

3.1	KRYPTON system architecture	22
3.2	LithOS system architecture	28
3.3	Hummingbird design overview	33
3.4	GPU-sharing techniques in the utilization–SLO space	34
4.1	KRYPTON: GPU fleet reduction and throughput accuracy . . .	42
4.2	LithOS inference-only: SLO attainment vs. throughput	44
4.3	LithOS hybrid inference-training: P99 latency and throughput .	45
4.4	Hummingbird single-GPU: SLO attainment and LP throughput	49
4.5	Hummingbird multi-GPU distributed scheduling	50
4.6	Hummingbird cross-GPU generalization (L40s, A100, H100) . .	52

List of Tables

1.1	Layered framing of three synthesized systems	5
2.1	Comparison of GPU sharing systems across key design dimensions.	15
4.1	KRYPTON experimental results	42
4.2	LithOS experimental results	47
4.3	Hummingbird experimental results	53

Chapter 1

Introduction

1.1 Background and Motivation

The past three years have witnessed a dramatic shift in the nature of GPU workloads running in cloud datacenters. Classical batch training jobs dominated GPU usage for most of the deep learning era, tolerating tens of minutes of end-to-end latency and making resource scheduling relatively straightforward. Today, however, the workload mix has fundamentally changed. Generative AI services—including large language model (LLM) inference [11, 20, 23, 28], text-to-speech (TTS) synthesis, and diffusion-based image generation [17]—account for an ever-growing fraction of GPU compute time, and they impose qualitatively different constraints on the scheduling infrastructure.

Unlike batch training, generative inference is inherently *iterative* and *interactive*. An LLM autoregressive decoding run [2, 10, 20, 24] issues hundreds of sequential forward passes, each reading and updating the KV cache and separated by host-side token sampling and stopping checks; a diffusion model [17] applies the same denoising U-Net dozens of times in a chain; a TTS system interleaves acoustic model inference with vocoder steps. Each iteration involves

a burst of GPU kernels followed by brief synchronization or host-device communication, creating a highly dynamic utilization profile that varies within a single request. From the scheduler’s perspective, this means that a request cannot be modeled as a single GPU kernel or even a fixed-length kernel sequence; its resource demand fluctuates unpredictably within tight end-to-end latency bounds.

In production deployments, this variability is compounded by *bursty request arrival* and *workload heterogeneity*. Real serving clusters must simultaneously handle latency-sensitive online requests (with P99 latency SLOs of 100–500 ms) and throughput-oriented batch jobs such as model fine-tuning or offline feature extraction. Providing dedicated GPUs for each workload class is prohibitively expensive: GPU utilization in practice often sits below 50% even in well-managed clusters [29], leaving significant capacity underutilized. Sharing GPUs across workload classes is therefore economically necessary, but doing so while preserving latency guarantees for priority traffic is technically difficult.

The root cause of this difficulty lies in the mismatch between what modern GPU sharing primitives offer and what production workloads require. Three deficiencies stand out.

Coarse preemption granularity. Commercial GPU hardware—NVIDIA A100, H100, and similar—does not expose an interrupt mechanism at the kernel level. Once a thread block begins executing on a Streaming Multiprocessor (SM), the hardware offers no way to stop it mid-flight. Preemption therefore happens between full kernel launches, meaning that a long matrix-multiplication kernel (which may run for 2–10 ms) cannot be interrupted when a high-priority request arrives. This forces temporal-sharing schedulers to wait for the incumbent kernel to drain before handing the GPU to an urgent task.

Weak isolation in spatial sharing. When multiple processes or contexts share the GPU concurrently—via NVIDIA Multi-Process Service (MPS) or similar mechanisms—they compete for LLC bandwidth, DRAM bandwidth, and shared memory, even if their compute allocation is separated. The resulting interference is workload-dependent and hard to predict, making it difficult to offer throughput or latency SLOs in a multi-tenant setting.

Inflexible hardware partitioning. NVIDIA Multi-Instance GPU (MIG) provides strong hardware isolation by partitioning the A100/H100 into up to seven fixed-size instances with dedicated SMs, memory slices, and bandwidth. However, MIG profiles are coarse-grained and cannot be changed while workloads are running, preventing the dynamic adaptation needed when request rates or workload phases change within a service window.

1.2 The Layered Scheduling Problem

We argue that addressing these three deficiencies simultaneously requires a **layered systems approach**. No single mechanism can provide the combination of runtime compatibility, fine execution control, and fast emergency preemption that production generative serving demands. Instead, the problem decomposes into three complementary concerns that map naturally to different positions in the software stack.

The *entrance control* layer must answer: how are heterogeneous workloads admitted to a shared GPU cluster with compatible interfaces, strong isolation, and controllable performance envelopes? This layer must operate above individual kernel launches and below the application framework, making it the natural home for virtualization and resource orchestration.

The *execution abstraction* layer must answer: once a workload is admitted,

at what granularity is scheduling performed within the GPU, and who decides how many compute resources each workload gets at each moment? This layer must operate at the granularity of kernel atoms or sub-kernel units to overcome head-of-line blocking and enable dynamic right-sizing.

The *emergency preemption* layer must answer: when a high-priority request arrives while a low-priority kernel is executing, how quickly can the GPU be reclaimed without hardware support for mid-kernel interruption? This layer must pre-shape workloads so that the worst-case waiting time is bounded at the microsecond scale.

This report synthesizes three recent systems—KRYPTON [29], LithOS [4], and Hummingbird [9]—that collectively address these three layers. KRYPTON operates at the entrance-control layer, using kernel-space command-buffer interception to construct isolated virtual GPU (IGPU) instances that span both spatial (MIG) and temporal (time-slice) dimensions. LithOS operates at the execution-abstraction layer, introducing TPC-level scheduling and transparent kernel atomization to replace coarse, kernel-granularity execution management with a software-defined GPU OS. Hummingbird operates at the emergency-preemption layer, pre-splitting low-priority kernels into bounded-duration quanta and harvesting idle bubbles to keep the GPU productive while guaranteeing that a high-priority task can acquire the GPU within hundreds of microseconds.

Table 1.1 summarizes the three-layer framing and maps each system to its layer.

Table 1.1: Layered framing of the three synthesized systems.

Paper	Layer	Key Mechanism		Limitation Addressed
KRYPTON [29]	Entrance control	Kernel-space interception	IGPU	User-space remoting brittleness; weak isolation
LithOS [4]	Execution abstraction	TPC scheduling + kernel atoms		Coarse kernel granularity; head-of-line blocking
Hummingbird [9]	Emergency preemption	Split-kernel quanta + bubble harvesting		Millisecond preemption latency on closed-source GPUs

1.3 Contributions

The primary contribution of this report is a unified architectural synthesis that explains how KRYPTON, LithOS, and Hummingbird compose into a coherent, closed-loop GPU scheduling stack. Specifically, this report:

1. **Provides a three-layer decomposition** of the GPU scheduling problem for iterative generative workloads, demonstrating that entrance control, execution abstraction, and emergency preemption are separable concerns that must nevertheless be co-designed.
2. **Analyzes the design rationale and mechanisms** of each system in depth, including the KRYPTON IGPU abstraction and spatio-temporal orchestration, the LithOS TPC scheduler and kernel atomization framework, and the Hummingbird kernel splitter and bubble-harvesting runtime.
3. **Synthesizes experimental evidence** from all three papers to show that the layered approach reduces GPU fleet requirements by 32.1%, lowers

P99 tail latency by up to $13\times$ compared with NVIDIA MPS, and achieves $9.7\times$ higher SLO attainment than state-of-the-art spatial sharing—collectively demonstrating the necessity of all three layers.

4. **Identifies inter-layer dependencies and design tensions**, showing how the layers are both complementary and composable, and discussing the residual limitations that future unified GPU operating environments must address.

1.4 Structure of the Report

The remainder of this report is organized as follows. Chapter 2 surveys the prior literature on GPU sharing, covering temporal sharing, spatial sharing, SLO-oriented preemption systems, and GPU virtualization approaches, and positions KRYPTON, LithOS, and Hummingbird within this landscape. Chapter 3 presents the proposed layered architecture, describing each system’s design goals, core mechanisms, and layer-specific contributions, followed by a unified analysis of how the three layers compose into a closed-loop scheduling stack. Chapter 4 details the experimental methodology, presents evaluation results for each layer, discusses cross-system comparisons, and identifies limitations and future research directions, before concluding the report.

Chapter 2

Related Work

GPU scheduling has been an active research area for over a decade, driven by the growing economic importance of GPU clusters in both cloud and on-premise deployments. We survey the major approaches, organized by the primary scheduling dimension they exploit, and then position the three systems synthesized in this report within the broader landscape.

2.1 Temporal GPU Sharing

The simplest form of GPU sharing is *temporal multiplexing*: workloads receive exclusive access to the GPU in turns, with a scheduler enforcing turn-taking. This approach has a long history in production GPU management, dating to NVIDIA’s time-slicing API introduced in its enterprise driver stack.

NVIDIA Time-Slicing and MPS. NVIDIA’s default GPU time-sharing divides GPU access into fixed-length time slices at the process granularity. While straightforward, time-slicing suffers from high context-switch overhead (tens of milliseconds for large models) and does not allow fine-grained priority enforcement because the scheduler can only preempt between CUDA API calls

at the host side, not between kernel launches on the device.

NVIDIA Multi-Process Service (MPS) [29] reduces launch overhead by funneling multiple client processes through a single CUDA context, but it does not fundamentally solve the preemption latency problem: if a long-running kernel is in flight when a high-priority task arrives, MPS provides no mechanism to interrupt it. Moreover, running under a shared CUDA context means that a crash or memory error in one tenant can corrupt shared GPU state, weakening fault isolation.

PipeSwitch. PipeSwitch [1] demonstrated fast GPU context switching for DNN inference by pipelining memory transfer with computation: while a new model is being loaded into GPU memory over PCIe, the previous model is still running its final iterations. This approach achieves context switch times of tens of milliseconds—fast for batch DNN tasks, but still orders of magnitude too slow for the microsecond preemption targets required by latency-sensitive generative serving.

Gandiva. Microsoft Research’s Gandiva [26] introduced time-sliced GPU sharing with workload migration between GPUs, exploiting the feedback between training loss curves and scheduling decisions to improve cluster-level efficiency. Gandiva focuses on training job scheduling rather than mixed inference/training co-location and does not address the latency SLO requirements of interactive serving.

More broadly, cluster-level schedulers such as Tiresias [7], Gavel [14], Themis [13], HiveD [30], and Pollux [16] address fairness, heterogeneity-awareness, and goodput optimization across GPU clusters, but operate at the job-level granularity and do not tackle the within-GPU preemption latency that is central to latency-sensitive generative serving.

The common limitation of all temporal sharing systems is the *long preemp-*

tion gap: because hardware does not support mid-kernel interruption, the worst-case waiting time for a high-priority task is bounded by the longest kernel execution time of co-located workloads, which can reach several milliseconds for compute-bound matrix operations.

2.2 Spatial GPU Sharing

Spatial sharing allows multiple workloads to execute concurrently on different partitions of the GPU’s compute resources, reducing average waiting time at the cost of potential interference.

CUDA Streams and MPS Spatial Partitioning. The simplest form of spatial sharing uses CUDA streams with different priorities. NVIDIA’s Cooperative Kernel Launch and priority streams allow multiple kernels to co-reside on the GPU if they do not fully saturate compute resources. However, interference in L2 cache, DRAM bandwidth, and the memory interconnect is uncontrolled, making it difficult to offer throughput or latency guarantees to individual tenants.

NVIDIA Multi-Instance GPU (MIG). Introduced with the A100, MIG provides hardware-enforced partitioning: the GPU can be sliced into up to seven instances, each with a dedicated set of SMs, a private L2 cache partition, and a dedicated HBM memory slice. MIG instances are fully isolated—a memory error in one instance cannot affect others—and each instance exposes a standard CUDA interface. However, MIG profile shapes are coarse (7 fixed options for A100) and cannot be changed while workloads are running, preventing the runtime adaptation that heterogeneous workloads demand. Furthermore, the SM granularity of MIG instances means that small latency-sensitive inference jobs must either be over-provisioned (receiving a larger MIG slice than they

need) or under-provisioned (receiving fewer SMs than optimal for their batch size).

GPUlet. GPUlet [3] extended MIG-based sharing with a dynamic allocation policy that adjusts the MIG profile assignment across requests, attempting to improve utilization relative to static MIG assignment. However, because MIG reconfiguration requires draining and restarting running workloads, GPUlet still operates at coarse time granularity and cannot react to within-request phase changes.

Fractional GPU. Fractional GPU explored fine-grained spatial partitioning below the MIG level by controlling which SMs each process’s kernels may use, enforced through CUDA kernel launch interception. While it demonstrated that sub-kernel-launch spatial isolation is achievable in software, it was evaluated only on simple workloads and does not scale to the complex multi-framework deployments characteristic of modern serving clusters.

The fundamental limitation of spatial sharing is *bandwidth interference*: even when compute units are partitioned, the L2 cache, DRAM, and NVLink remain shared. For memory-bandwidth-intensive workloads—LLM decoding, for instance, is heavily memory-bound [6, 11]—spatial partitioning of SMs does not eliminate performance interference from co-located tenants.

2.3 SLO-Oriented and Priority-Aware Scheduling

A third line of work explicitly models latency SLOs and implements priority-aware scheduling to protect latency-sensitive workloads.

REEF. REEF [8] introduced GPU preemption via context reset: when a high-priority task needs the GPU, the low-priority task is preempted by resetting the GPU context and discarding in-progress work. REEF demonstrated that con-

text reset preemption is practical on commercial GPUs and achieves better SLO attainment than purely temporal or spatial sharing. However, the preemption latency of REEF is bounded by the time required to drain the device-side command queue and perform the context reset, which takes on the order of several milliseconds—still insufficient for the sub-millisecond SLO targets of TTS or interactive LLM serving. In addition, REEF’s context reset approach discards partially completed computation, increasing the amount of redundant work for the low-priority task.

Orion. Orion [21] targets the co-location of DNN inference and training tasks on a single GPU. It uses spatial multiplexing at SM granularity, assigning different SM ranges to the high-priority and low-priority tasks. Orion’s key insight is that the SM utilization of most DNN kernels is less than 100%, leaving room for a co-located workload without significant compute interference. However, experiments reported in Hummingbird [9] show that Orion achieves HP SLO attainment of less than 22.8% across eight high-priority / low-priority workload combinations, indicating that spatial partitioning alone is insufficient when the low-priority workload consumes significant memory bandwidth.

TGS (Transparent GPU Sharing). TGS [25] provides transparent kernel-level time-slicing by intercepting CUDA kernel launches at the driver level and inserting scheduling decisions between kernel boundaries. TGS can enforce time quotas at finer granularity than process-level time-slicing, but its preemption points are still tied to inter-kernel boundaries, meaning that long-running kernels still block priority switching for the duration of their execution.

Clipper and Nexus. Early inference serving systems such as Clipper [5] and Nexus [19] established latency-aware batching and SLO-driven request routing at the cluster level, demonstrating that ML serving requires dedicated scheduling infrastructure beyond general-purpose resource managers. However, both

systems treat the GPU as a black-box device and do not address within-GPU preemption or fine-grained resource partitioning.

AlpaServe. AlpaServe [12] exploits model parallelism as a statistical multiplexing mechanism for LLM serving, placing and parallelising collections of large models across distributed clusters to absorb bursty request arrivals within latency SLOs. While AlpaServe achieves up to $10\times$ higher request rates compared to model-parallel baselines, it operates at the cluster placement granularity and does not address within-GPU scheduling or preemption.

Baymax. Baymax addresses QoS in GPU sharing by predicting the performance interference caused by co-located workloads and using this prediction to decide placement and resource allocation. It models GPU memory hierarchy interference using machine learning models trained on workload characterization data. Baymax improves SLO compliance relative to unmanaged co-location, but relies on offline interference models that may not generalize to novel workload combinations.

2.4 GPU Virtualization and API Remoting

GPU virtualization approaches seek to present a standard GPU interface to applications while multiplexing the physical hardware across tenants.

API Remoting (rCUDA, Cricket). Early virtualization systems like rCUDA and Cricket intercept high-level CUDA API calls (e.g., `cudaMalloc`, `cudaLaunchKernel`) and forward them to a remote GPU server. API remoting enables GPU sharing across network boundaries, but its tight coupling to specific CUDA library versions makes it fragile: a minor CUDA update can break interception if the new library uses different internal APIs. Furthermore, API remoting cannot intercept non-CUDA runtimes (such as Vulkan or OpenCL), limiting its com-

patibility in heterogeneous clusters.

Antman. Alibaba’s Antman [27] provides production-grade GPU sharing for a mixed training-and-inference cluster. Antman uses a cooperation protocol between the GPU manager and individual ML frameworks to implement memory oversubscription and dynamic time-slice adjustment. While effective for workloads that can be modified to cooperate with Antman’s runtime, it requires framework-side instrumentation and cannot share GPUs across unmodified or closed-source applications.

vGPU and Virtual Machine Approaches. NVIDIA GRID and similar vGPU products partition a physical GPU into virtual GPU instances that can be assigned to virtual machines. These approaches prioritize compatibility and strong isolation for cloud VMs but typically expose a fixed-size virtual GPU with no support for dynamic resource reallocation based on runtime demand.

The key limitation shared by most virtualization approaches is that interception at the user-space library level—whether CUDA API, OpenCL, or a higher-level framework API—is inherently fragile, because it must track and replicate all behaviors of a complex, version-specific software stack. KRYPTON’s innovation of moving interception into kernel space directly addresses this limitation.

2.5 GPU Operating System Abstractions

The most recent direction in GPU scheduling treats the GPU as a first-class managed resource, analogous to a CPU in a traditional operating system, and introduces OS-like abstractions for resource allocation and isolation.

PTask. PTask [18] introduced a dataflow task model for GPU computation, managing GPU resources through an OS-level graph scheduler. It demon-

strated that OS-level control of GPU resource allocation improves throughput and reduces interference, but its performance benefits depend on adapting applications to the PTask programming model.

Kernel Preemption via Savepoints. Several research systems [9] have explored inserting software-defined preemption points within GPU kernels by modifying the kernel source code to periodically check a preemption flag. This approach can achieve preemption granularity at the thread-block level, but requires kernel source code access—infeasible for closed-source ML framework kernels such as cuBLAS, cuDNN, or vendor-compiled TensorRT engines.

Transparent Kernel Splitting. The insight underlying Hummingbird’s kernel splitter—that kernels can be split into smaller execution quanta without modifying kernel source code, by controlling the number of thread blocks dispatched per launch—builds on observations from the thread-block-level scheduling literature. The novelty of Hummingbird lies in making this splitting transparent (no source modification needed), formally bounding the resulting preemption delay, and combining it with a bubble-harvesting policy that converts the overhead of splitting into useful throughput.

2.6 Positioning of KRYPTON, LithOS, and Hummingbird

Table 2.1 summarizes the design dimensions of the systems discussed above and positions KRYPTON, LithOS, and Hummingbird within this design space.

Taken together, the prior work reveals three persistent gaps. First, no existing system achieves *both* runtime-agnostic compatibility and fine-grained performance control—user-space interception is fragile, and hardware mechanisms (MIG) are too coarse. KRYPTON addresses this by moving intercep-

Table 2.1: Comparison of GPU sharing systems across key design dimensions.

System	Sharing Mode	Intercept Level	Preemption Granularity	SLO	Compatibility
MPS	Spatial	Runtime	Kernel	×	CUDA only
MIG	Spatial	Hardware	Static	~	CUDA only
REEF	Temporal	Driver	Context	✓	CUDA
Orion	Spatial	Runtime	Kernel	~	CUDA
TGS	Temporal	Driver	Kernel	~	CUDA
PipeSwitch	Temporal	Runtime	Model	×	PyTorch
Antman	Temporal	Framework	Request	~	Modified FW
KRYPTON	Spatio-temporal	KMD	Time-slice	✓	CUDA + Vulkan
LithOS	Spatial	Driver	Kernel atom	✓	CUDA (transparent)
Hummingbird	Temporal	Driver	Split-kernel	✓	CUDA (transparent)

tion into the kernel mode driver. Second, no existing mechanism schedules at sub-kernel granularity transparently—kernel splitting approaches either require source modification or operate at the coarse kernel boundary. LithOS addresses this through transparent driver-level interposition with kernel atomization. Third, no prior system achieves sub-millisecond preemption latency on closed-source GPUs while maintaining competitive best-effort throughput—REEF’s context-reset approach discards work and incurs context-switch overhead; Orion’s spatial partitioning leaves HP SLO attainment far below production requirements. Hummingbird addresses this through proactive kernel splitting and bubble harvesting. The three systems thus occupy complementary positions in the design space, motivating the layered synthesis presented in Chapter 3.

Chapter 3

Proposed Methods

This chapter presents the three systems—KRYPTON, LithOS, and Hummingbird—as successive layers of a coherent GPU scheduling architecture. For each system we describe the design goals, key observations motivating the design, core mechanisms, and the layer’s contribution to the overall stack. Section 3.4 then provides a unified interpretation of how the layers compose and interact.

3.1 KRYPTON: Control at the GPU Entrance

3.1.1 Design Goals and Motivating Observations

KRYPTON begins from two practical observations that limit the effectiveness of prior GPU virtualization and sharing approaches in production clusters [29].

The first observation is that *user-space API remoting is too fragile for heterogeneous production clusters*. Systems that intercept high-level CUDA library calls (e.g., at the cuDNN or cuBLAS level) are tightly coupled to specific library versions. A minor CUDA update—changing an internal data structure layout, introducing a new lazy-initialization path, or adding an asynchronous mem-

ory manager—can silently break the interception layer, requiring significant engineering effort to maintain across CUDA releases. More fundamentally, user-space interception cannot handle non-CUDA runtimes: rendering workloads using Vulkan or OpenCL, which are increasingly common in multi-tenant GPU clusters, pass through a completely different software path and cannot be captured by a CUDA-specific shim.

The second observation is that *isolation is weak when multiple tenants share a single CUDA context*. Systems such as MPS aggregate multiple client processes into one CUDA context to reduce kernel launch overhead. While this improves throughput, it means that a buggy or adversarial tenant can corrupt shared GPU state, read another tenant’s memory, or trigger GPU resets that affect all co-located workloads. True multi-tenancy requires per-tenant isolation that MPS cannot provide without hardware support.

3.1.2 The IGPU Abstraction

KRYPTON’s central contribution is the **IGPU** (Isolated GPU) abstraction. An IGPU bundles three resources into a unified, isolated execution environment:

- **Spatial allocation:** A MIG instance that provides a dedicated slice of SMs, L2 cache, and HBM memory bandwidth. The spatial allocation is selected from the discrete set of MIG profiles supported by the target hardware (e.g., 1g.5gb, 2g.10gb, 3g.20gb, 4g.20gb, 7g.40gb on an A100 40GB). This gives KRYPTON hardware-enforced memory isolation between tenants.
- **Temporal quota:** A time-slice fraction of the MIG instance’s compute capacity allocated to the IGPU. Multiple IGPUs can be mapped onto the same MIG instance and receive time-sliced access, allowing finer-grained allocation than MIG profiles alone would permit.

- **Memory envelope:** A GPU memory allocation cap enforced by the interception layer, preventing memory overuse from affecting co-tenants.

By combining a spatial dimension (MIG profile selection) with a temporal dimension (time-slice quota), KRYPTON escapes the rigidity of pure MIG assignment and the interference of pure time-sharing: two IGPUs can share the same MIG instance temporally, or two IGPUs can occupy different MIG instances and potentially execute concurrently.

3.1.3 Kernel-Space Interception

The key enabler of KRYPTON’s runtime-agnostic compatibility is *kernel-space interception* [29]. Rather than hooking into user-space libraries, KRYPTON intercepts GPU command buffers at the GPU kernel mode driver (KMD) level—the lowest software layer through which all GPU commands must pass before reaching the hardware, regardless of whether they originate from CUDA, Vulkan, OpenCL, or any other runtime.

At this level, KRYPTON interposes between the OS GPU driver and the hardware, capturing:

- Kernel launch packets (which encode the thread block configuration, shared memory size, and kernel function pointer),
- Memory copy commands (host-to-device, device-to-host, device-to-device),
- Synchronization primitives (stream events, device-side barriers), and
- Context management operations (context creation, destruction, GPU reset).

By operating at this level, KRYPTON can enforce time-slice quotas (by delaying or deferring kernel launch packets when a tenant’s quota is exhausted)

and memory limits (by intercepting allocation requests) without any knowledge of the higher-level runtime or ML framework in use. The same interception layer handles CUDA 10.x, CUDA 12.x, and Vulkan applications transparently, because all of them eventually submit commands through the KMD interface.

3.1.4 System Components and Operation

KRYPTON comprises four cooperating components, illustrated in Figure 3.1 [29].

Offline Profiler. Before deployment, the profiler characterizes each workload’s performance under different (MIG profile, time-quota) combinations. Profiling identifies the Pareto-optimal operating points—the minimum MIG profile and minimum time quota that allow the workload to meet its throughput or latency target—and stores these in a performance model used by the orchestrator.

Kernel-Space Interception Module. The interception module, running as a kernel-mode component, captures all GPU command submissions from every tenant process. It enforces the time quota allocated to each IGPU by tracking accumulated GPU time within each scheduling epoch and blocking further submissions when the quota is exhausted. Enforcement is done at command-buffer boundaries to minimize latency jitter.

Performance Feedback Controller. Because workload behavior can drift over time (e.g., changing batch sizes, changing input lengths in LLM decoding), KRYPTON employs a closed-loop feedback controller that periodically measures actual throughput or latency for each IGPU and adjusts its time quota accordingly. The controller attempts to minimize the gap between measured and target performance while respecting the aggregate quota constraint across all IGPUs on a MIG instance. In practice, KRYPTON achieves an average through-

put relative error of only 0.9% with the feedback controller active [29], compared to 5.3% without feedback.

Spatio-Temporal Orchestrator. The orchestrator solves the placement problem: given a set of IGPU requests (each with a profiled performance model and an SLO target), find an assignment of IGPUs to (MIG instance, time-quota) pairs that maximizes GPU utilization while satisfying all SLO constraints. KRYPTON formalizes this as a two-stage bin-packing problem: Stage 1 selects the MIG profile for each IGPU (packing IGPUs into MIG instances while respecting SM and memory constraints), and Stage 2 assigns time quotas given the Stage 1 spatial assignment (packing temporal quotas within each MIG instance). The two-stage formulation makes the otherwise NP-hard joint optimization tractable.

3.1.5 Layer Contribution

From the perspective of the layered architecture, KRYPTON solves the *admission and isolation* problem. By the time a workload’s commands enter the GPU hardware, KRYPTON has already ensured that: (1) the workload’s interface is compatible regardless of its runtime, (2) it is isolated from co-tenants at both the spatial (MIG) and memory levels, and (3) its time quota is calibrated to bring actual performance within a fraction of a percent of its SLO target. The execution-layer (LithOS) and emergency-preemption-layer (Hummingbird) systems can then assume that workloads are admitted under controlled, predictable conditions.

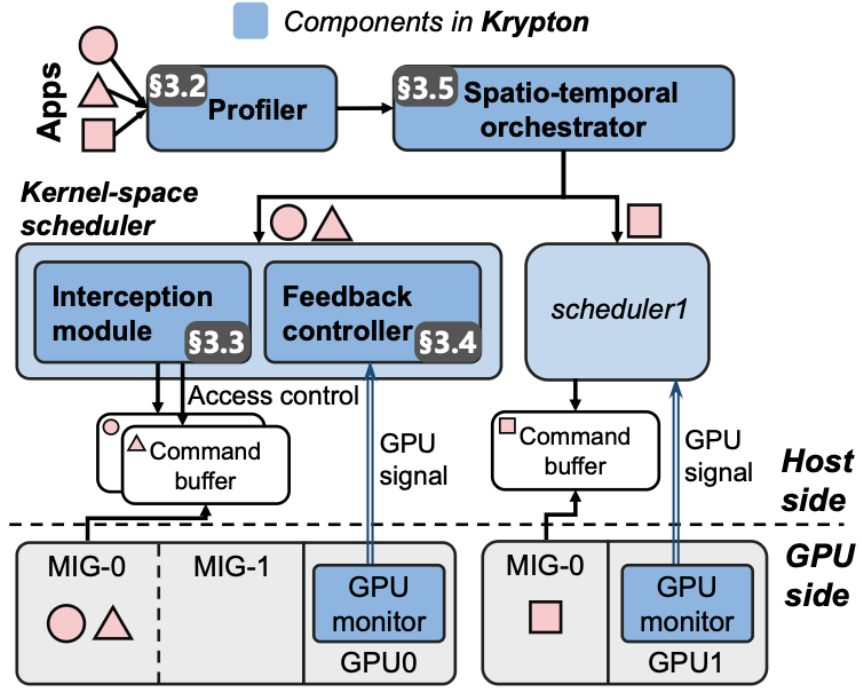


Figure 3.1: KRYPTON system architecture overview [29]. The offline profiler characterizes workload performance; the kernel-space interception module enforces time quotas; the feedback controller adjusts quotas based on observed performance; the spatio-temporal orchestrator places IGPUs via two-stage bin-packing.

3.2 LithOS: Execution Abstraction Inside the GPU

3.2.1 Design Goals and Motivating Observations

LithOS begins from a diagnosis complementary to KRYPTON’s: *even when admission and isolation are correctly handled, the scheduling granularity of existing mechanisms is too coarse for fine-grained multi-tenant ML workloads* [4].

Existing GPU schedulers—including MIG, MPS, and KRYPTON’s time-slicing—operate at the granularity of whole kernels or whole requests. When a large matrix-multiplication kernel is in flight, a co-located workload with a smaller kernel must wait. This *head-of-line blocking* inflates tail latency: a short, latency-sensitive inference step must queue behind a long training gradient computation that, from the hardware’s perspective, is indistinguishable from a sequence of identical thread block groups. Furthermore, because neither the framework nor the driver knows how many TPCs the workload actually needs for its current kernel atom (as opposed to the maximum it could use), workloads are routinely given more compute resources than necessary—wasting SMs that could be given to co-located tenants.

LithOS addresses these problems by introducing OS-like abstractions that give the scheduler explicit control over the mapping between software work units (kernel atoms) and hardware resources (TPCs).

3.2.2 GPU Hardware Background

To understand LithOS’s mechanisms, it is helpful to recall the structure of a modern NVIDIA GPU. An A100 GPU contains 108 Streaming Multiprocessors (SMs), organized into 54 Texture Processing Clusters (TPCs) of 2 SMs each. When a kernel is launched, the GPU’s thread block scheduler distributes thread

blocks across available SMs. Crucially, thread blocks assigned to a given SM run to completion before new thread blocks from another kernel can occupy that SM: the hardware does not support preemption of executing thread blocks on commercial A100/H100 GPUs.

LithOS schedules at TPC granularity. A TPC can be logically assigned to one workload at a time (preventing thread blocks from another workload from being scheduled onto its SMs), and TPCs can be reassigned between kernel atom launches. This gives LithOS the ability to dynamically resize a workload’s compute allocation between atoms.

3.2.3 TPC-Level Scheduling and TPC Stealing

The first key mechanism of LithOS is the **TPC Scheduler** [4]. Unlike static MIG allocation or whole-GPU MPS sharing, the TPC Scheduler maintains a per-epoch assignment of TPCs to active workloads and dynamically updates this assignment between kernel atom launches.

TPC stealing is the primary mechanism for improving utilization. When workload A ’s next kernel atom is ready to launch but uses fewer TPCs than its maximum allocation (due to right-sizing, described below), LithOS identifies idle TPCs and offers them to co-located workload B . If B can absorb additional TPCs (i.e., its next atom has not yet been dispatched), it receives the additional compute without waiting for a full scheduling quantum. When A ’s next atom arrives and needs more TPCs, the stolen TPCs are returned.

TPC stealing is transparent to both workloads: neither A nor B needs to be modified. LithOS implements TPC stealing by intercepting kernel launches and adjusting the grid dimensions and SM masks passed to the hardware, so that each kernel atom sees exactly its allocated set of TPCs.

3.2.4 Kernel Atomization

The second key mechanism is **kernel atomization** [4]: the transparent partitioning of a monolithic GPU kernel into a sequence of *atoms*, each consisting of a bounded subset of the original kernel’s thread blocks.

Formally, given a kernel with N total thread blocks and an atom size k , atomization replaces a single kernel launch with $\lceil N/k \rceil$ sequential launches, each dispatching k thread blocks. The atom size k is chosen by the right-sizing algorithm (Section 3.2.5) to balance two objectives: (1) keeping each atom short enough that LithOS can reassign TPCs between atoms at reasonable frequency, and (2) keeping atoms long enough that the amortized per-atom scheduling overhead remains negligible.

Kernel atomization provides two direct benefits. First, it bounds head-of-line blocking: a co-located workload waits at most for the current atom to complete (a few hundred microseconds), not the entire multi-millisecond kernel. Second, it creates scheduling opportunities: between atoms, LithOS can insert a different workload’s atoms, implement TPC stealing, or adjust frequency without requiring any hardware interrupt.

LithOS implements atomization transparently by interposing at the GPU driver level. When a workload issues a kernel launch with N thread blocks, LithOS intercepts the launch and re-issues it as a series of k -block launches, invisible to the calling framework. This approach works for any kernel whose thread blocks are independent—a property that holds for all matrix multiply, convolution, attention, and most other ML computation kernels.

3.2.5 Hardware Right-Sizing

The third mechanism is **hardware right-sizing**: for each kernel atom, LithOS estimates the minimum number of TPCs needed to complete the atom within the workload’s latency budget, and uses only that many TPCs, freeing the remainder for co-located workloads [4].

Right-sizing relies on a lightweight performance model that maps (kernel type, atom size, TPC count) to execution time. This model is built from offline profiling and online performance counter feedback. During scheduling, LithOS queries the model to determine the minimum TPC allocation $t_{\min}$ such that the atom completes before its per-atom deadline, and allocates exactly $t_{\min}$ TPCs. If the workload has positive latency slack (its end-to-end latency is below its SLO target), LithOS can reduce allocation further to a “right-sized” value, releasing additional TPCs to co-tenants.

Across a representative workload mix, right-sizing reduces average GPU capacity consumption by 26% and up to 51% in peak reduction, with less than 4% increase in P99 latency [4]—a substantial efficiency gain without meaningful SLO degradation.

3.2.6 Transparent Power Management

Building on right-sizing, LithOS implements **transparent Dynamic Voltage and Frequency Scaling (DVFS)** [4]. When a workload has latency slack and right-sizing has already reduced its TPC count, LithOS can additionally reduce the GPU clock frequency. Lower frequency reduces GPU power consumption quadratically with clock ratio, at the cost of proportionally slower kernel execution. LithOS selects the lowest frequency at which the right-sized TPC count can still complete each atom within its per-atom deadline.

DVFS is transparent: the workload sees no change in its programming interface, and the only observable effect is slower kernel execution, which is compensated by the latency slack budget. In practice, LithOS achieves average GPU energy savings of 26% (up to 46% per-workload) with a mean P99 latency increase of only 7% [4].

Figure 3.2 shows how the four mechanisms—TPC scheduling, kernel atomization, hardware right-sizing, and power management—are layered between unmodified application frameworks and the GPU device driver.

3.2.7 Layer Contribution

LithOS operates *beneath* the KRYPTON admission layer and *above* the bare GPU hardware. Given that KRYPTON has already ensured that each workload is admitted with compatible interfaces and isolated memory, LithOS’s job is to ensure that the GPU’s compute resources are allocated at the finest practical granularity during execution. The combination of TPC stealing, kernel atomization, right-sizing, and DVFS collectively transforms the GPU from a device that runs one monolithic kernel at a time to a flexibly partitioned engine that can simultaneously serve multiple workloads at their respective SLO points while minimizing wasted compute and energy.

3.3 Hummingbird: Emergency Preemption and Bubble Harvesting

3.3.1 Design Goals and Motivating Observations

Hummingbird targets the scenario that neither KRYPTON nor LithOS fully addresses: the arrival of a *time-critical* high-priority (HP) request while a *best-effort*

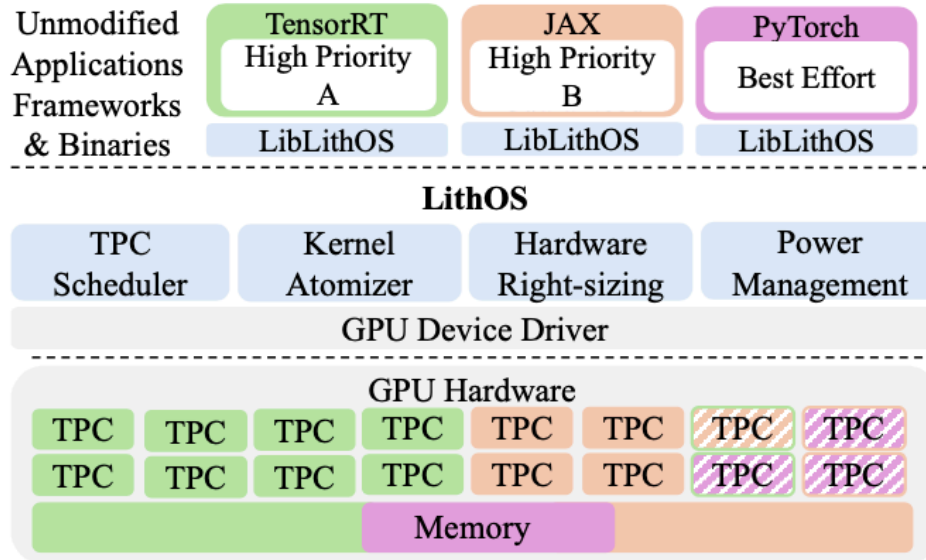


Figure 3.2: LithOS system architecture [4]. Applications from multiple frameworks (TensorRT, JAX, PyTorch) attach to the GPU through per-process LibLithOS shims. The LithOS layer sits between the application shims and the GPU device driver, providing four core services: a TPC Scheduler for fine-grained compute allocation, a Kernel Atomizer for transparent kernel splitting, Hardware Right-sizing to match TPC count to workload demand, and Power Management for dynamic frequency scaling. At the hardware level, TPCs are partitioned among co-located workloads (green: high-priority A; orange: high-priority B; purple: best-effort) with separate memory allocations.

low-priority (LP) workload is executing a long kernel [9].

Even with LithOS’s kernel atomization, atoms of memory-bandwidth-intensive LP workloads (such as LLM fine-tuning with Llama-70B as the LP task) can span hundreds of milliseconds per atom if the batch size or sequence length is large. Reducing atom size enough to bound preemption delay at the microsecond scale would require extremely small atoms—each containing only a handful of thread blocks—and would degrade LP throughput due to excessive per-launch overhead. Furthermore, LithOS’s right-sizing focuses on efficient resource allocation for co-existing workloads, not on guaranteeing that an urgently arriving HP request gets the GPU within a fixed latency bound.

Hummingbird’s design is therefore guided by two key observations [9]:

1. **Thread blocks complete quickly.** Profiling of GPU kernels from LLM inference, training, and convolution workloads shows that 99.999% of thread blocks complete within 400 microseconds, regardless of the total kernel duration. This is because each thread block performs a bounded amount of work (a fixed tile of matrix multiplication, for instance), and the variation in thread block runtime is small. This observation means that if the system can bound the *number* of thread blocks dispatched per kernel launch, it automatically bounds the worst-case waiting time for a high-priority task.
2. **HP workloads generate idle GPU bubbles.** Iterative generative workloads—particularly LLM autoregressive decoding—naturally create gaps in GPU utilization. Between decoding iterations, the host must gather output tokens, run sampling, check stopping criteria, and prepare the next iteration’s input, creating bubbles of 1–50 ms during which the GPU is idle. Additionally, within each iteration, PCIe or NVLink transfers, host-side softmax or sampling operations, and KV-cache memory allocation create

smaller bubbles of 10–200 μ s. Both types of bubble represent wasted GPU capacity that could be given to LP work.

3.3.2 Kernel Splitting

The core mechanism of Hummingbird is **kernel splitting** [9]: pre-transforming each LP kernel into a sequence of *split-kernels*, each launching a bounded number S of thread blocks per hardware dispatch.

Unlike LithOS’s kernel atomization (which is driven by TPC scheduling policies), Hummingbird’s splitting is driven by the preemption delay bound: the split size S is chosen so that a single split-kernel completes within a target preemption delay δ (e.g., 200 μ s). Given the thread-block completion distribution measured by the offline profiler, S is the maximum number of thread blocks such that the 99.999th percentile completion time of S blocks does not exceed δ .

The **kernel splitter** performs this analysis offline: it profiles each LP kernel’s thread-block execution time distribution, computes the appropriate split size S , and instruments the LP workload to replace each kernel launch of N thread blocks with a loop over $\lceil N/S \rceil$ split-kernel launches of S blocks each. Crucially, the split-kernel loop includes a priority check between launches: if a HP request has arrived, the LP process immediately yields the GPU (by simply not issuing the next split-kernel launch) and does not resume until the HP work completes.

The splitting approach does not require modifying the LP kernel source code: it operates at the level of the CUDA launch API (`cudaLaunchKernel`), replacing a single large launch call with a series of smaller ones. This transparency extends to compiled kernels (TensorRT engines, cuBLAS kernels) that cannot be instrumented at the source level.

3.3.3 Bubble-Aware Runtime Scheduling

The **runtime scheduler** in Hummingbird is responsible for detecting idle GPU bubbles and filling them with LP split-kernels, and for enforcing HP preemption when needed [9].

Hummingbird classifies GPU bubbles into two types based on their expected duration:

Large bubbles occur when the HP workload is waiting for I/O, host-side computation, or network communication between iterations. These bubbles last tens to hundreds of milliseconds—long enough to fit many LP split-kernels and make meaningful progress on the LP task. The runtime scheduler detects large bubbles by monitoring HP GPU stream activity: when no HP kernels have been submitted for more than a threshold time τ_{large} (e.g., 1 ms), the scheduler declares a large bubble and begins dispatching LP split-kernels.

Small bubbles occur within an HP iteration, between individual HP kernel launches, during memory transfers, or during host-side operations. These bubbles last 10–200 μs —too short to initiate a full LP kernel, but sufficient for a single or a few split-kernels. The runtime scheduler uses tight-loop polling on HP stream activity to detect small bubbles and opportunistically fills them with the smallest feasible LP split.

The scheduler must balance two conflicting objectives: (1) maximizing LP throughput by filling as many bubbles as possible, and (2) ensuring that the HP task can acquire the GPU within δ of its arrival. Hummingbird achieves both by construction: because each split-kernel is bounded to complete within δ , the HP task need never wait more than δ for the current split to drain, regardless of which type of bubble is being filled.

3.3.4 Hierarchical Memory Offloading

For LP workloads with large memory footprints—particularly large-model fine-tuning or decoding tasks such as Llama-70B—GPU memory itself becomes a preemption bottleneck: even after the LP kernel is split and stopped, the LP model weights and KV cache occupy GPU HBM and cannot be immediately reused by an HP request that requires different model weights [9].

Hummingbird addresses this through **hierarchical memory offloading**, which moves LP model parameters and activations from GPU HBM to CPU memory over NVLink when an HP task arrives. The offloading is hierarchical in two senses. First, different memory regions are offloaded at different priorities: the LP KV cache (which can be recomputed from the prompt) is evicted before model parameters (which must be preserved to resume LP computation). Second, within model parameters, layers that are not needed for the next forward pass (deep layers, for long-horizon decoding) are offloaded before layers that are needed immediately.

NVLink’s high bandwidth (e.g., 600 GB/s aggregate for NVLink 3.0 on A100 SXM4) makes hierarchical offloading fast: even a 70B-parameter model (140 GB at FP16) can be partially offloaded at rates that keep memory preemption delay within the same order of magnitude as compute preemption delay.

Figure 3.3 illustrates how the kernel splitter, runtime scheduler, and memory manager cooperate, while Figure 3.4 positions Hummingbird against prior sharing techniques in the utilization-vs-SLO plane.

3.3.5 Layer Contribution

Hummingbird operates as the *emergency preemption* layer beneath KRYPTON’s isolation guarantees and orthogonally to LithOS’s execution-time scheduling.

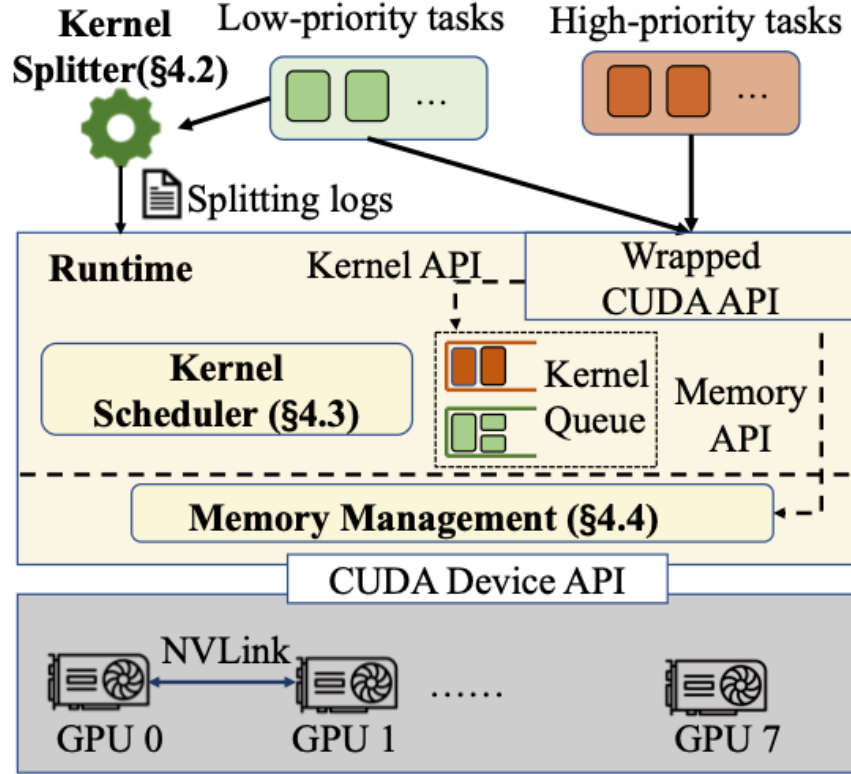


Figure 3.3: Hummingbird design overview [9]. The kernel splitter profiles LP kernels and produces bounded split-kernels; the runtime scheduler detects large and small GPU bubbles and fills them with split-kernels; the hierarchical memory manager offloads LP memory over NVLink when HP tasks require additional HBM.

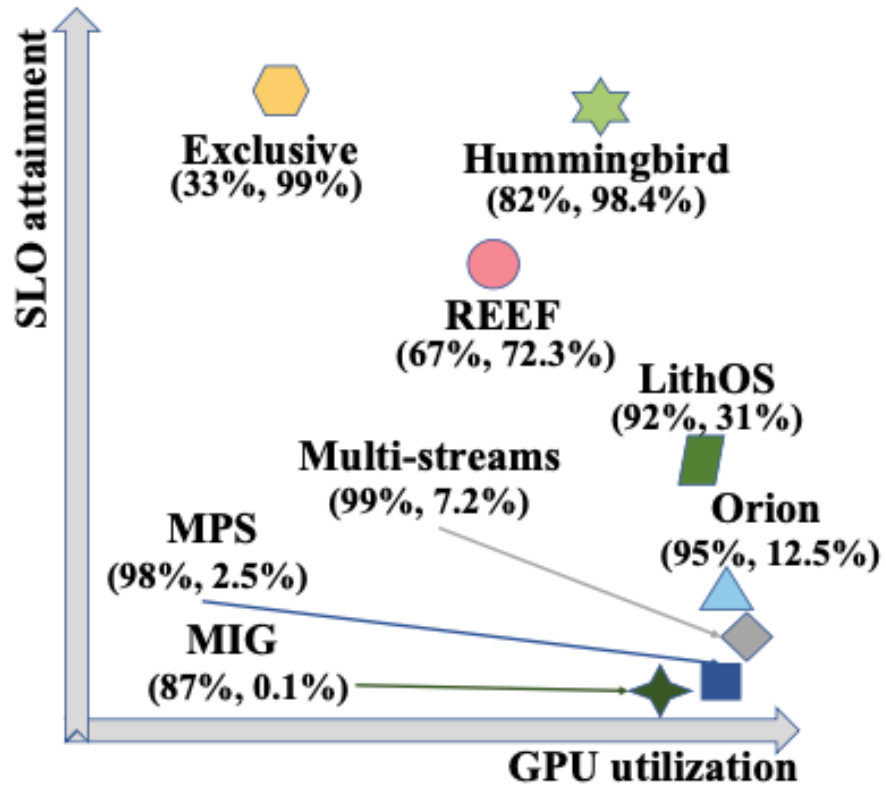


Figure 3.4: Taxonomy of GPU-sharing techniques in the utilization-SLO space [9]. The x-axis represents GPU utilization and the y-axis represents SLO attainment for HP tasks. Hummingbird achieves a Pareto-dominant position: 82% utilization with 98.4% HP SLO attainment, compared to exclusive (high SLO, low utilization) and spatial sharing (high utilization, low SLO).

While LithOS improves the efficiency of normal-case multi-tenant execution, Hummingbird provides the *worst-case* guarantee: regardless of what LP workload is executing, a newly arrived HP request will acquire the GPU within a bounded latency. This guarantee is independent of workload type, batch size, or sequence length, because it is enforced at the split-kernel granularity by construction.

3.4 A Unified Layered Interpretation

3.4.1 How the Three Layers Compose

The three systems form a coherent layered architecture in which each layer’s output is the next layer’s input:

KRYPTON (Control Plane) is the outermost layer. It accepts heterogeneous workloads from multi-tenant clients, assigns each an IGPU with appropriate spatial (MIG) and temporal (time-slice) resources, and presents each workload with a stable, isolated virtual GPU interface. From the perspective of LithOS and Hummingbird, all workloads have already been admitted and isolated; no cross-tenant compatibility issues remain.

LithOS (Execution Plane) operates within each KRYPTON IGPU (or independently on a GPU not managed by KRYPTON). It subdivides the execution time and compute resources of admitted workloads at the finest practical granularity—TPC atoms—continuously rebalancing TPC allocation and clock frequency to maximize aggregate throughput and energy efficiency while respecting per-workload latency budgets. LithOS is the “steady-state” efficiency layer; it governs resource allocation during normal operation.

Hummingbird (Interrupt Plane) operates as an orthogonal emergency layer. Rather than scheduling between co-equal tenants (as KRYPTON and LithOS

do), it enforces a strict priority ordering: HP work always wins, and LP work yields within a microsecond-scale deadline. Hummingbird does not replace LithOS’s scheduling during non-emergency periods; it adds a preemption guarantee on top.

3.4.2 Inter-Layer Interactions and Design Tensions

The three layers are designed to be composable, but several inter-layer interactions warrant attention.

KRYPTON + LithOS: KRYPTON’s time-slice boundaries create natural epochs at which LithOS can adjust TPC allocations. If a workload’s time slice expires, KRYPTON suspends its command buffer submissions, and LithOS can reclaim its TPCs for other workloads. This cooperation means that KRYPTON’s coarse-grained temporal control and LithOS’s fine-grained execution management operate at compatible timescales without interfering.

LithOS + Hummingbird: LithOS’s kernel atomization reduces atom size to tens of milliseconds; Hummingbird’s split-kernel mechanism further reduces execution quanta to hundreds of microseconds. In a combined deployment, LithOS’s right-sized atoms serve as the input to Hummingbird’s splitter: the splitter further subdivides each LithOS atom into split-kernels of size $S \ll k$. This two-level subdivision cleanly separates the concerns of resource efficiency (LithOS’s right-sizing minimizes TPC waste) and preemption latency (Hummingbird’s splitting minimizes HP waiting time).

KRYPTON + Hummingbird: KRYPTON’s kernel-space interception layer must be aware of Hummingbird’s split-kernel loop to avoid counting each split-kernel launch against the IGPU’s quota as if it were an independent request. In practice, KRYPTON’s quota accounting operates at the GPU time level (wall-clock time the GPU is executing on behalf of this IGPU), so multiple

split-kernel launches within a single bubble are correctly accumulated into a single quota charge.

3.4.3 Why All Three Layers Are Necessary

The layered decomposition reveals a clear argument for why each layer is indispensable:

- **Without KRYPTON**, LithOS and Hummingbird inherit the compatibility and isolation problems of user-space interception. A heterogeneous cluster mixing CUDA and Vulkan workloads, or mixing CUDA versions, cannot safely share GPUs without kernel-space admission control.
- **Without LithOS**, even a correctly isolated GPU (via KRYPTON) wastes compute on head-of-line blocking and over-provisioned SMs. Steady-state efficiency—serving multiple workloads near their individual SLO points simultaneously—requires fine-grained execution scheduling.
- **Without Hummingbird**, tail latency for HP tasks remains bounded by LP kernel duration. Even with LithOS’s atomization, LP atoms of hundreds of milliseconds can occasionally occur in memory-intensive workloads, violating the strict microsecond preemption targets of interactive generative serving.

The three layers thus form a *closed loop*: KRYPTON enables safe admission of heterogeneous workloads; LithOS maximizes efficiency during normal execution; Hummingbird provides strict tail-latency guarantees in emergency scenarios. No subset of two layers can achieve all three properties simultaneously.

Chapter 4

Experiments and Conclusion

This chapter presents the experimental evidence for each layer of the proposed architecture. We first describe the evaluation methodology and workloads used in each paper (Section 4.1), then present results for KRYPTON (Section 4.2), LithOS (Section 4.3), and Hummingbird (Section 4.4), followed by a cross-system comparative analysis (Section 4.5), discussion of limitations and future directions (Section 4.6), and a conclusion (Section 4.7).

4.1 Experimental Setup

The three papers span different hardware configurations, workload types, and evaluation scopes, reflecting their different layer concerns. We summarize each below.

KRYPTON Evaluation Setup. KRYPTON is evaluated on NVIDIA A100 40GB and RTX 4090 GPUs. The evaluation covers a heterogeneous workload mix including CUDA-based AI inference and training tasks (ResNet-50, BERT, GPT-2 under various batch sizes) and Vulkan-based rendering workloads, deployed in separate Linux containers to simulate a realistic multi-tenant clus-

ter environment. KRYPTON is compiled against CUDA 12.1 and Vulkan 1.3 drivers. Baseline comparisons include: (1) *temporal sharing* (exclusive round-robin time-slicing), (2) *best-fit MIG* (static MIG profile assignment using best-fit bin packing without time multiplexing), and (3) *GPUlet-MIG* (the GPUlet approach applied to MIG instances). Ten representative co-location test cases are evaluated, each consisting of two to four co-located workloads with different throughput targets.

LithOS Evaluation Setup. LithOS is implemented in Rust and evaluated on a single A100 SXM4 GPU with 108 SMs and 40 GB HBM2. A wide range of ML frameworks is used without modification: Triton [22] (for custom ML kernels), TensorRT (for optimized inference engines), TensorRT-LLM (for LLM inference), ONNX Runtime (for cross-framework portability), and PyTorch [15] (for training). Evaluated workloads include ResNet-50 inference, BERT-Large inference, GPT-2 inference and training, and Llama-3 8B decoding. Baselines include MPS, MIG (static profile assignment), TGS [25], REEF [8], and Orion [21]. Three experimental scenarios are evaluated: (1) inference-only multitenancy (two HP inference tasks sharing a GPU), (2) hybrid inference-training multitenancy (one HP inference task co-located with one LP training task), and (3) resource efficiency (right-sizing and DVFS with a single workload).

Hummingbird Evaluation Setup. Hummingbird is evaluated on three GPU generations: NVIDIA L40s, A100 80GB SXM4, and H100. Distributed experiments use up to 16 A100 GPUs ($2 \times$ DGX A100 nodes connected via NVLink). HP workloads include diverse inference tasks: CNN inference (ResNet-50, VGG-16), LLM inference (Llama-8B, Mistral-7B, Llama-70B), and TTS inference. LP workloads include DNN training (GPT-2, ResNet-50), LLM fine-tuning (Llama-7B), and DDP training (GPT-2 across multiple GPUs). Eight HP $\times$ LP co-location combinations are evaluated on each hardware platform. Baselines include Orion [9],

REEF [9], LithOS [9], and exclusive execution (HP only, no LP). The primary metrics are HP SLO attainment (fraction of HP requests meeting their P99 latency target) and LP throughput relative to REEF.

4.2 KRYPTON: Compatibility, Isolation, and Cluster Efficiency

4.2.1 GPU Fleet Reduction

The headline result of KRYPTON is its ability to serve the same workload set with **32.1% fewer GPUs** than temporal sharing, 23.1% fewer than best-fit MIG, and 20.5% fewer than GPUlet-MIG, across ten heterogeneous co-location test cases [29]. This reduction arises directly from KRYPTON’s ability to combine MIG-level spatial sharing with fine-grained temporal multiplexing: rather than assigning each workload an entire MIG instance (which wastes spatial capacity when the workload is idle) or a pure time slice (which wastes temporal capacity when the workload is compute-light), KRYPTON finds the most efficient (MIG-profile, time-quota) pair for each workload through its two-stage bin-packing orchestrator.

The fleet reduction translates directly to capital cost savings: in a cluster of 1000 GPUs, KRYPTON reduces the required fleet by over 300 units while maintaining the same aggregate service capacity.

4.2.2 Throughput Accuracy and QoS

KRYPTON’s feedback controller achieves average throughput relative error of only **0.9%** (compared to the SLO target), versus 5.3% without the feedback loop [29]. This tight control is critical for production deployments where oper-

ators specify throughput or latency SLOs and expect them to be reliably met. The feedback controller’s ability to compensate for workload drift (changing batch sizes, input lengths) without requiring workload restarts is a key practical advantage.

For Vulkan rendering workloads—a class of applications that no prior GPU sharing system could manage—KRYPTON achieves average relative performance error of less than 3.4% [29], demonstrating the cross-runtime generality of kernel-space interception. Latency QoS violations (HP requests exceeding their target latency) average only 1.23% across all test cases, confirming that KRYPTON’s admission control keeps priority traffic within its SLO envelope.

4.2.3 Production Deployment

A key indicator of practical relevance is KRYPTON’s production deployment: the system has been running in a production GPU cluster for over one year at the time of publication [29]. This real-world deployment provides evidence of KRYPTON’s stability, compatibility, and operational manageability at scale—properties that laboratory evaluations cannot fully validate.

Figure 4.1 summarizes the headline results of the KRYPTON evaluation: across ten heterogeneous co-location test cases, KRYPTON consistently requires the smallest GPU fleet to meet the same SLO targets, while keeping per-workload throughput tightly clamped to the configured target.

Table 4.1 presents the full experimental results. KRYPTON strictly dominates or ties all baselines across all reported metrics. The combination of fleet reduction, throughput accuracy, Vulkan compatibility, and production deployment confirms that KRYPTON successfully addresses the entrance-control layer’s requirements.

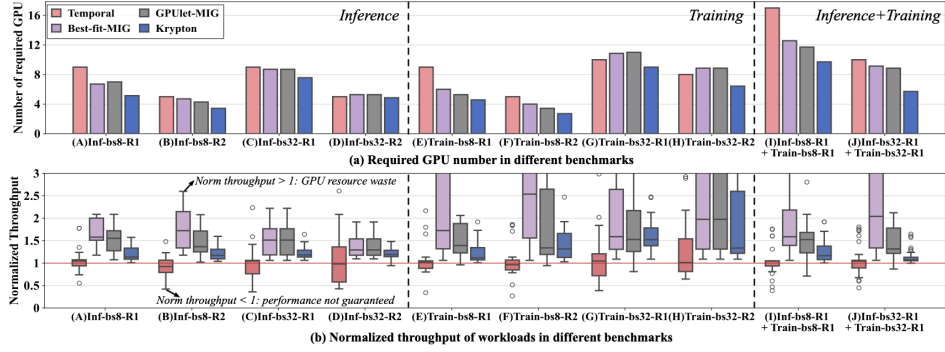


Figure 4.1: KRYPTON experimental results [29]. (a) Number of GPUs required across 10 test cases: KRYPTON consistently requires the fewest GPUs, reducing the total by 32.1% compared to temporal sharing. (b) Normalized throughput of co-located workloads under KRYPTON: throughput tracks the target (normalized value 1.0) most closely, demonstrating the effectiveness of feedback control.

Table 4.1: KRYPTON experimental results [29]. Platform: A100 40 GB & RTX 4090; CUDA 12.1, Vulkan 1.3.

Metric	Result
GPU fleet reduction	Best across all baselines: Temporal 32.1%, Best-fit-MIG 23.1%, GPUlet-MIG 20.5%
Throughput relative error (CUDA)	0.9% vs. 5.3% (no feedback)
Throughput relative error (Vulkan)	<3.4%
Latency QoS violation rate	1.23%
Multi-GPU throughput error	1.8%
Production deployment	>1 year (ongoing)

4.3 LithOS: Fine-Grained Scheduling and Energy Efficiency

4.3.1 Inference-Only Multitenancy

In the inference-only co-location scenario, two HP inference workloads share a single A100 SXM4 GPU. LithOS achieves **100% SLO attainment** while maintaining aggregate throughput at $1.0\times$ (the single-workload baseline) [4]. This result is remarkable because it demonstrates that LithOS can serve two inference workloads simultaneously without sacrificing either’s SLO or their combined throughput—a result that no prior baseline achieves.

By contrast, NVIDIA MPS achieves only 45% SLO attainment: the interference from shared memory bandwidth and the inability to prioritize short inference kernels over long co-located kernels cause frequent SLO violations. MIG achieves 100% SLO but at only 71% of the baseline throughput—because the static MIG partition cannot reclaim idle SM resources from one workload for use by the other. TGS achieves 84% SLO at roughly $0.78\times$ throughput, reflecting the trade-off inherent in time-sliced kernel-level scheduling.

LithOS reduces P99 tail latency by $13\times$ compared to MPS and by $4\times$ compared to the best prior baseline [4], while improving aggregate HP goodput by $1.3\times$ —demonstrating that fine-grained TPC stealing and kernel atomization provide a qualitative improvement over prior sharing mechanisms. Figure 4.2 visualizes the SLO-vs-throughput Pareto frontier for this scenario.

4.3.2 Hybrid Inference-Training Multitenancy

The hybrid scenario co-locates one HP inference workload with one LP training workload—the most common production co-location pattern, where spare

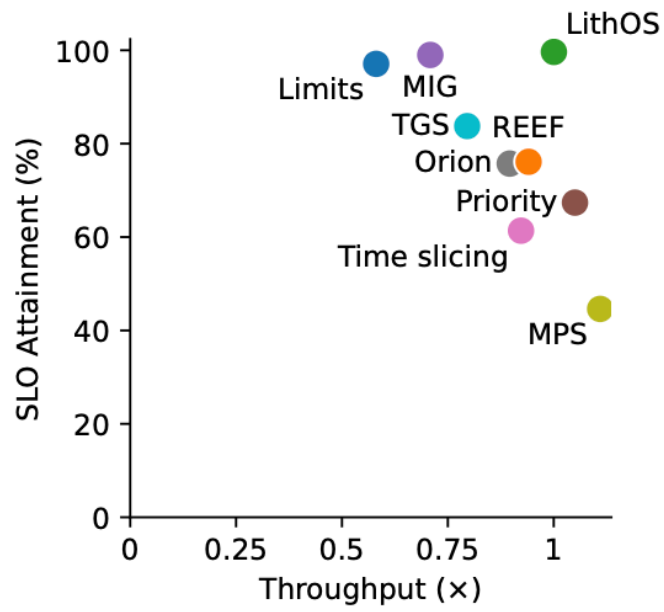


Figure 4.2: LithOS inference-only multitenancy: SLO attainment vs. aggregate throughput [4]. LithOS is the only system achieving 100% SLO attainment at full throughput (1.0 $\times$). MPS trades SLO for throughput; MIG trades throughput for SLO; TGS achieves neither.

capacity during inference idle time is filled with training jobs.

LithOS achieves P99 tail latency of $1.2\times$ **the single-workload ideal**, compared to $5.83\times$ for MPS, $1.41\times$ for TGS, and $2.89\times$ for REEF [4]. The $1.2\times$ ratio means LithOS adds only 20% tail latency overhead over having the GPU exclusively, even while simultaneously running a training workload—a result that comes within reach of the hardware limit. LithOS also achieves $4.7\times$ lower tail latency than MPS and $1.35\times$ higher aggregate throughput than the best state-of-the-art baseline. Figure 4.3 presents the per-model breakdown across five inference workloads.

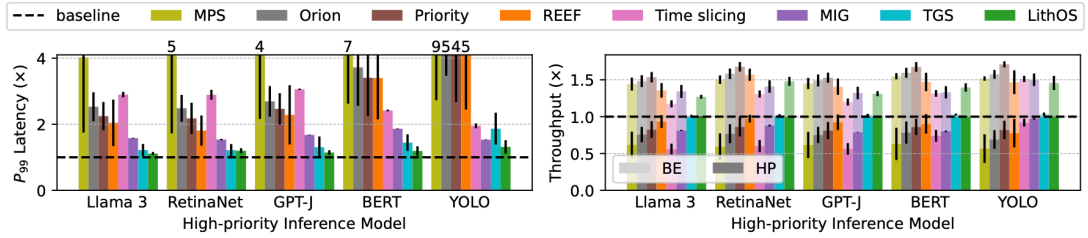


Figure 17. Hybrid multitancy: (a) P_{99} service latency and (b) aggregate throughput.

Figure 4.3: LithOS hybrid inference-training multitancy [4], with five HP inference models (Llama 3, RetinaNet, GPT-J, BERT, YOLO) co-located with an LP training task. (Left) P99 service latency normalized to the single-workload baseline: LithOS keeps tail latency within $1.2\times$ of ideal, far better than all baselines. (Right) Aggregate throughput (HP+LP) normalized to the single-workload baseline: LithOS improves aggregate throughput by $1.35\times$ over the best SoTA baseline.

4.3.3 Resource Efficiency: Right-Sizing and DVFS

LithOS’s right-sizing reduces average GPU SM capacity consumption by 26% and up to 51% per workload, with a maximum P99 latency increase of only

4% [4]. When combined with DVFS, LithOS further reduces GPU energy consumption by an average of 26% (up to 46%), with a maximum P99 latency increase of 7%.

These efficiency improvements are additive: a workload cluster that adopts both right-sizing and DVFS can expect roughly a 25% reduction in SM provisioning requirements and a 25% reduction in GPU energy bills, with latency SLOs maintained. The interposition overhead introduced by LithOS is only 4%, and kernel atomization adds less than 1% additional overhead—demonstrating that the scheduling sophistication of LithOS comes at minimal runtime cost.

Table 4.2 presents the complete LithOS experimental results across all three scenarios.

4.4 Hummingbird: SLO Protection and Bubble Harvesting

4.4.1 Single-GPU Priority Scheduling

On a single A100 GPU with 8 HP×LP workload combinations, Hummingbird achieves HP SLO attainment of $\sim 99\%$ —nearly matching the exclusive (HP-only) baseline of $99\%+$ —while simultaneously running an LP workload [9]. Orion, by comparison, achieves less than 22.8% SLO attainment across the same scenarios; REEF achieves $2.7\times$ Orion’s result (still far below Hummingbird); LithOS achieves $1.8\times$ Orion’s.

Compared to exclusive execution, Hummingbird’s co-location of an LP workload reduces HP SLO attainment by less than 1%—an extraordinary result that shows the LP task is almost invisible to the HP workload from a latency perspective. At the same time, Hummingbird achieves $1.9\times$ LP throughput com-

Table 4.2: LithOS experimental results [4]. Platform: 1×A100 SXM4 (108 SMs, 40 GB HBM).

Metric	LithOS	Baselines
<i>Inference-only multitenancy</i>		
SLO attainment	100%	MPS 45%, TGS 84%, MIG 100%
Aggregate throughput	1.0×	MPS 1.11×, MIG 0.71×
P99 latency improvement	13× vs. MPS; 4× vs. best SotA	—
HP goodput improvement	1.3×	over MPS
<i>Hybrid inference-training</i>		
P99 tail latency (vs. ideal)	1.2×	MPS 5.83×, REEF 2.89×, TGS 1.41×
Aggregate throughput	1.35×	TGS 1.0×
<i>Right-sizing & power management</i>		
Capacity savings (avg / max)	26% / 51%	P99 latency increase <4%
Energy savings (avg / max)	26% / 46%	P99 latency increase <7%
Overhead	< 4%	interposition; atomization adds <1%

pared to REEF and 82% GPU utilization (versus 67% for REEF), demonstrating that the efficiency gained through bubble harvesting is real and substantial. Figure 4.4 reports the per-scenario breakdown for both metrics.

4.4.2 Memory-Intensive Workloads

The memory-intensive scenario (Llama-70B as the LP workload) is the most challenging for priority scheduling: the LP model’s 140 GB (FP16) footprint requires aggressive memory offloading when HP requests arrive, and its memory-bandwidth-intensive decoding kernels can saturate DRAM even when only a fraction of SMs are active.

Hummingbird achieves $5.6\times$ HP SLO attainment versus REEF and $4.2\times$ LP throughput versus REEF in this scenario [9]. Both Orion and LithOS achieve less than 12% SLO attainment—indicating that spatial partitioning alone cannot isolate the HP workload from the memory bandwidth consumption of Llama-70B. Hummingbird’s hierarchical memory offloading over NVLink is specifically designed to address this case, and the results confirm its effectiveness.

4.4.3 Multi-GPU Distributed Scheduling

Distributed training and inference introduce additional scheduling complexity: the LP and HP workloads span multiple GPUs and communicate over NVLink or InfiniBand, so preemption of one GPU must be coordinated with all other GPUs in the LP task’s distributed group.

Hummingbird scales to $16\times$ A100 GPUs with Llama-405B as the HP workload and GPT-2 DDP training as the LP workload, achieving $9.7\times$ HP SLO attainment versus REEF and $3.3\times$ LP throughput versus REEF [9]. The $9.7\times$ improvement in SLO attainment is the largest reported result in the Hummingbird

evaluation, reflecting the compounding benefit of Hummingbird’s approach in a setting where prior systems’ inter-GPU synchronization overhead further degrades HP latency. Figure 4.5 pairs the SLO and throughput metrics with the corresponding preemption-delay reduction.

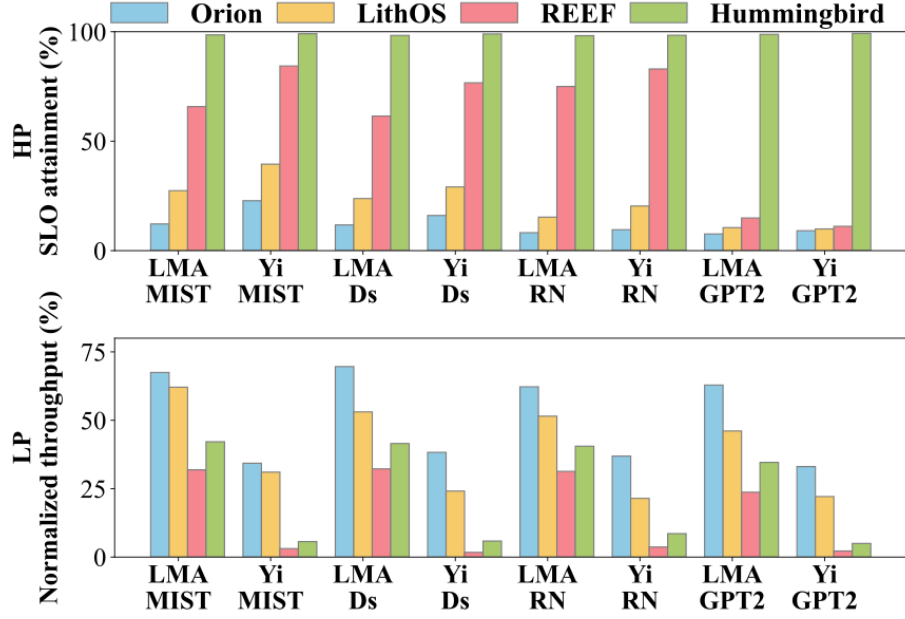


Figure 4.4: Hummingbird single-GPU results [9]. (Top) HP SLO attainment across 8 HP×LP co-location scenarios: Hummingbird achieves $\sim 99\%$ attainment on all combinations, within 1% of exclusive execution, while Orion stays below 23%. (Bottom) Normalized LP throughput on the same scenarios: Hummingbird delivers $1.9\times$ higher LP throughput than REEF on average through bubble harvesting; Orion’s higher LP throughput is a consequence of its failure to preserve HP SLO (top panel).

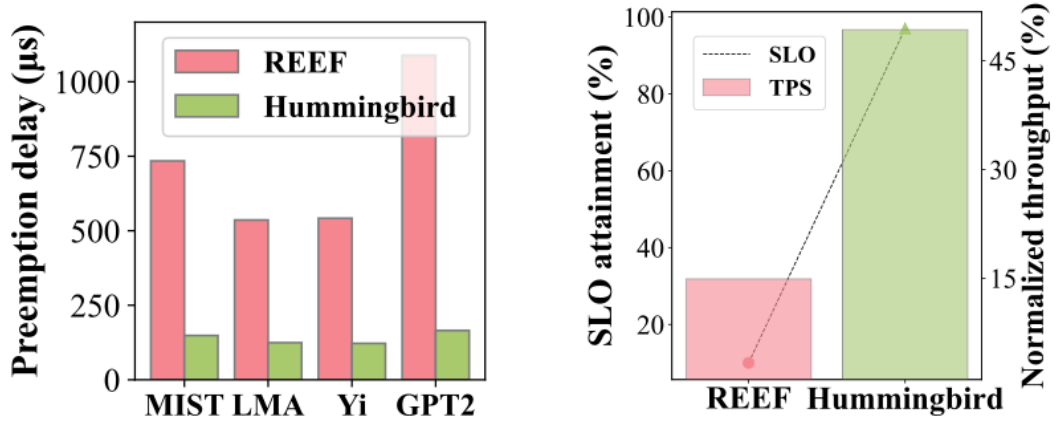


Figure 4.5: Hummingbird multi-GPU distributed scheduling results [9], on $16 \times A100$ ($2 \times DGX A100$) with Llama-405B as HP and GPT-2 DDP as LP. (Left) Preemption delay across four LP workloads: Hummingbird stays in the 100–175 μs range while REEF takes 540–1080 μs . (Right) HP SLO attainment and normalized LP throughput: Hummingbird achieves $9.7\times$ SLO improvement and $3.3\times$ LP throughput over REEF.

4.4.4 Cross-GPU Generalization

A key practical concern is whether Hummingbird’s performance is specific to a particular GPU generation or transfers across hardware. Hummingbird is evaluated on L40s, A100, and H100 GPUs with Llama-8B and Mistral-7B as HP workloads, reporting P99 Time-Per-Output-Token (TPOT) reduction relative to Orion and REEF [9].

Results show P99 TPOT reductions of **39%** on L40s, **31%** on A100, and **13.3%** on H100. The smaller reduction on H100 reflects the H100’s faster NVLink 4.0 interconnect (which allows REEF’s context reset to complete faster, narrowing the gap) but Hummingbird still achieves the best TPOT on all three platforms. Figure 4.6 reports both HP P99 TPOT and LP throughput across the three GPU generations.

4.4.5 Preemption Delay and Overhead

Hummingbird’s average preemption delay—the time between an HP request arriving and the GPU being available to the HP task—is **121–165 μ s** across evaluated workloads [9], compared to $4.3\text{--}6.6\times$ longer for REEF. The total overhead of the kernel splitting and scheduling mechanism is only 1.3% of GPU time, and the SLO impact of bubble consolidation is less than 0.7%—demonstrating that Hummingbird’s safety guarantees come at minimal efficiency cost.

Table 4.3 summarizes the complete Hummingbird experimental results.

4.5 Comparative Analysis

Taken together, the three systems address the GPU scheduling problem from complementary angles and achieve improvements that no single system could

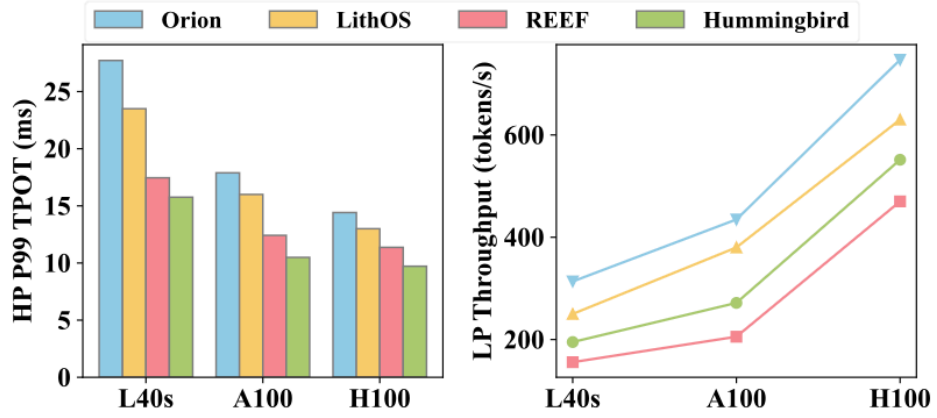


Figure 4.6: Hummingbird cross-GPU generalization [9]. (Left) HP P99 Time-Per-Output-Token (TPOT) across L40s, A100, and H100 GPUs: Hummingbird consistently achieves the lowest HP tail latency on every platform. (Right) LP throughput in tokens/s on the same platforms: Hummingbird preserves substantial LP throughput while still meeting HP SLOs—the higher LP throughput of Orion and LithOS comes at the cost of much worse HP latency (left panel). Together, the two panels confirm Hummingbird’s hardware-agnostic effectiveness.

Table 4.3: Hummingbird experimental results [9]. Platforms: L40s, A100 80 GB SXM4, H100; up to 16 GPUs.

Metric	Hummingbird	Baselines
<i>Single GPU (A100, 8 HP×LP combinations)</i>		
HP SLO attainment	~ 99%	Exclusive 99%+; REEF 2.7× Orion; Orion <22.8%
SLO vs. exclusive	< 1% drop	—
LP throughput	1.9 × REEF	GPU util. 82% vs. 67% (REEF)
<i>Memory-intensive (Llama-70B LP)</i>		
HP SLO attainment	5.6 × REEF	Orion & LithOS both <12%
LP throughput	4.2 × REEF	—
<i>Multi-GPU (16×A100, Llama-405B + GPT-2)</i>		
HP SLO attainment	9.7 × REEF	—
LP throughput	3.3 × REEF	—
<i>Cross-GPU (Llama-8B + Mistral-7B, L40s / A100 / H100)</i>		
P99 TPOT reduction	13–39%	lower than Orion & REEF on all three GPUs
LP throughput	1.25 × REEF	—
<i>Overhead</i>		
Preemption delay	121–165 μ s	REEF 4.3–6.6× longer
Runtime overhead	1.3% splitting; < 0.7% SLO impact	—

provide alone.

Dimension 1: Compatibility. KRYPTON is uniquely distinguished by its cross-runtime compatibility. LithOS and Hummingbird both operate at the CUDA driver level and inherit CUDA-only scope; adding KRYPTON’s kernel-space interception beneath them would extend the combined architecture’s compatibility to Vulkan and future non-CUDA runtimes.

Dimension 2: Isolation. KRYPTON provides hardware-enforced memory isolation via MIG and prevents cross-tenant GPU context corruption. LithOS provides logical TPC isolation at the scheduling level but relies on software enforcement rather than hardware barriers. Hummingbird provides temporal isolation under priority inversion but does not address spatial isolation between equal-priority tenants.

Dimension 3: Scheduling granularity. The three systems span three decades of scheduling timescale. KRYPTON operates at the time-slice level (tens to hundreds of milliseconds per epoch); LithOS operates at the kernel-atom level (hundreds of microseconds to tens of milliseconds per atom); Hummingbird operates at the split-kernel level (tens to hundreds of microseconds per split). Together, they provide scheduling points at all three timescales, enabling a scheduler to react to workload changes at the appropriate granularity.

Dimension 4: Resource efficiency. KRYPTON improves cluster-level efficiency (32.1% GPU fleet reduction). LithOS improves compute efficiency (26% GPU capacity savings) and energy efficiency (26% energy savings) within a single GPU. Hummingbird improves GPU utilization (82% vs. 67% for REEF) through bubble harvesting. Stacking all three layers would yield fleet-level, per-GPU compute, and per-GPU utilization improvements simultaneously.

Dimension 5: SLO protection. KRYPTON provides coarse-grained throughput SLO enforcement via feedback control (0.9% error). LithOS provides per-

workload P99 latency SLO enforcement via right-sizing and atomization. Hummingbird provides worst-case HP SLO guarantees via bounded split-kernel preemption. Together, they create a defense-in-depth SLO protection model that works at admission time (KRYPTON), execution time (LithOS), and emergency time (Hummingbird).

4.6 Limitations and Future Directions

Despite their collective strengths, the three systems have several important limitations that point to future research directions.

KRYPTON: Static MIG shapes. KRYPTON’s spatial allocation is constrained by the fixed MIG profile options available on NVIDIA hardware. When workload demand changes at a timescale shorter than the MIG reconfiguration window (which requires draining and restarting tenants), KRYPTON cannot adapt the spatial allocation without service disruption. A future system could address this by implementing software-defined spatial partitioning—enforcing SM allocation limits in software rather than relying on hardware MIG boundaries—at the cost of potentially weaker isolation guarantees.

LithOS: Kernel source independence limitations. While LithOS’s kernel atomization works for any kernel whose thread blocks are independent, a small class of GPU kernels uses cross-thread-block communication (e.g., via global memory atomics with fence ordering). Atomizing such kernels at arbitrary boundaries can produce incorrect results. LithOS must identify and exclude these kernels from atomization, potentially leaving them as head-of-line-blocking long kernels. Future work on software checkpointing or hardware-assisted kernel suspension could close this gap.

Hummingbird: Priority regime assumption. Hummingbird’s design as-

sumes a strict two-level priority hierarchy (HP vs. LP). In practice, production clusters often have more nuanced priority structures (e.g., gold, silver, bronze tiers with different SLO targets). Extending Hummingbird’s preemption guarantees to multi-tier priority hierarchies—where a silver task must yield to gold within δ but a bronze task may yield to silver within a looser bound—is an open design challenge.

The unified system gap. The most significant limitation is that the three systems were designed and evaluated independently. A production deployment that combines all three would require co-design at the interface boundaries—particularly the KRYPTON-LithOS interface (coordinating time-slice epochs with TPC atom boundaries) and the LithOS-Hummingbird interface (coordinating atom size with split-kernel size). Building and evaluating a unified GPU OS that integrates all three layers is an important direction for future work.

Hardware evolution. NVIDIA’s Hopper architecture (H100) and future generations are progressively introducing more hardware-level scheduling support, including faster context switching and more flexible SM partitioning. As hardware capabilities evolve, the software mechanisms of KRYPTON, LithOS, and Hummingbird may become partially redundant or require redesign to exploit new hardware primitives. Tracking this moving target is an ongoing systems research challenge.

4.7 Conclusion

This report has synthesized three recent GPU scheduling systems—KRYPTON, LithOS, and Hummingbird—into a unified three-layer architecture for iterative generative workloads. The central argument is that GPU scheduling for modern AI serving is a *layered systems problem*: no single mechanism can simultane-

ously address the requirements of runtime-agnostic compatibility, fine-grained execution efficiency, and worst-case tail-latency protection.

KRYPTON addresses the entrance-control layer by constructing isolated GPU (IGPU) abstractions through kernel-space command-buffer interception, enabling compatibility across CUDA and Vulkan runtimes and combining MIG spatial allocation with time-slice temporal allocation under feedback control. LithOS addresses the execution-abstraction layer by introducing TPC-level scheduling and transparent kernel atomization that replace coarse kernel-granularity management with fine-grained, software-defined GPU resource allocation, additionally providing hardware right-sizing and DVFS for energy efficiency. Hummingbird addresses the emergency-preemption layer by pre-shaping LP kernels into microsecond-bounded split-kernels and harvesting idle GPU bubbles during HP task execution, achieving near-exclusive-level HP SLO protection while maintaining competitive LP throughput.

The experimental results from all three papers, considered together, paint a consistent picture: the layered approach reduces GPU fleet requirements by 32.1%, cuts P99 tail latency by up to $13\times$ compared to NVIDIA MPS, and achieves $9.7\times$ higher HP SLO attainment than state-of-the-art spatial sharing in multi-GPU settings. These gains are mutually reinforcing rather than competing: fleet reduction (KRYPTON) and per-GPU efficiency (LithOS, Hummingbird) compound to deliver higher value from the same hardware investment.

The broader lesson for GPU systems research is that closed-source hardware is not a barrier to systems innovation. Even without hardware preemption hooks, without modifiable kernel sources, and without framework cooperation, it is possible to build scheduling mechanisms that achieve strong guarantees by choosing the right abstraction at the right software layer. The future trajectory is clear: a unified GPU operating environment in which isolated ad-

mission, fine-grained execution management, and emergency preemption are native, co-designed services will unlock the full potential of GPU infrastructure for the generative AI era.

Bibliography

- [1] Zhihao Bai, Zhen Zhang, Yibo Zhu, and Xin Jin. PipeSwitch: Fast pipelined context switching for deep learning applications. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020. [8](#)
- [2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. [1](#)
- [3] Seungbeom Choi, Sunho Lee, Yeonjae Kim, Jongse Park, Youngjin Kwon, and Jaehyuk Huh. Serving heterogeneous machine learning models on multi-GPU servers with spatio-temporal sharing. In *Proceedings of the 2022 USENIX Annual Technical Conference (ATC)*, 2022. [10](#)
- [4] P. H. Coppock, B. Zhang, E. H. Solomon, V. Kypriotis, L. Yang, B. Sharma, D. Schatzberg, T. C. Mowry, and D. Skarlatos. LithOS: An operating sys-

- tem for efficient machine learning on GPUs. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2025. [4](#), [5](#), [23](#), [24](#), [25](#), [26](#), [27](#), [28](#), [43](#), [44](#), [45](#), [46](#), [47](#)
- [5] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. Clipper: A low-latency online prediction serving system. In *Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2017. [11](#)
- [6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. [10](#)
- [7] Juncheng Gu, Mosharaf Chowdhury, Kang G. Shin, Yibo Zhu, Myeongjae Jeon, Junjie Qian, Hongqiang Liu, and Chuanxiong Guo. Tiresias: A GPU cluster manager for distributed deep learning. In *Proceedings of the 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2019. [8](#)
- [8] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022. [10](#), [39](#)
- [9] T. Hu, C. Wang, T. Cao, J. Qin, L. Chen, X. Xiao, J. Hu, H. Tian, S. Yan, H. Cui, Q. Chen, and T. Xie. Hummingbird: SLO-oriented GPU preemption at microsecond-scale. *arXiv preprint arXiv:2601.04071*, 2026. [4](#), [5](#), [11](#), [14](#), [29](#), [30](#), [31](#), [32](#), [33](#), [34](#), [39](#), [40](#), [46](#), [48](#), [49](#), [50](#), [51](#), [52](#), [53](#)

- [10] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. [1](#)
- [11] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, 2023. [1](#), [10](#)
- [12] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2023. [12](#)
- [13] Kshiteej Mahajan, Arjun Balasubramanian, Arjun Singhvi, Shivaram Venkataraman, Aditya Akella, Amar Phanishayee, and Shuchi Chawla. Themis: Fair and efficient GPU cluster scheduling. In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020. [8](#)
- [14] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. Heterogeneity-aware cluster scheduling policies for deep learning workloads. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020. [8](#)
- [15] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Brad-

- bury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. [39](#)
- [16] Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R. Ganger, and Eric P. Xing. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021. [8](#)
- [17] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. [1](#)
- [18] Christopher J. Rossbach, Jon Currey, Mark Silberstein, Baishakhi Ray, and Emmett Witchel. PTask: Operating system abstractions to manage GPUs as compute devices. In *Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP)*, 2011. [13](#)
- [19] Haichen Shen, Lequn Chen, Yuchen Jin, Liangyu Zhao, Bingyu Kong, Matthai Philipose, Arvind Krishnamurthy, and Ravi Sundaram. Nexus: A GPU cluster engine for accelerating DNN-based video analysis. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*, 2019. [11](#)
- [20] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley,

- Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019. [1](#)
- [21] Foteini Strati, Xianzhe Ma, and Ana Klimovic. Orion: Interference-aware, fine-grained GPU sharing for ML applications. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*, 2024. [11](#), [39](#)
- [22] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019. [39](#)
- [23] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023. [1](#)
- [24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [1](#)
- [25] Bingyang Wu, Zili Zhang, Zhihao Bai, Xuanzhe Liu, and Xin Jin. Transparent GPU sharing in container clouds for deep learning workloads. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2023. [11](#), [39](#)
- [26] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng,

- Hanyu Zhao, Quanlu Zhang, Fan Yang, and Lidong Zhou. Gandiva: Introspective cluster scheduling for deep learning. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018. [8](#)
- [27] Wencong Xiao, Shiru Ren, Yong Li, Yang Zhang, Pengyang Hou, Zhi Li, Yihui Feng, Wei Lin, and Yangqing Jia. AntMan: Dynamic scaling on GPU clusters for deep learning. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020. [13](#)
- [28] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-based generative models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022. [1](#)
- [29] S. Zhang, A. Xu, Q. Chen, H. Zhao, W. Cui, Z. Wang, Y. Li, L. Xiao, and M. Guo. Efficient performance-aware GPU sharing with compatibility and isolation through kernel space interception. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, 2025. [2](#), [4](#), [5](#), [8](#), [17](#), [19](#), [20](#), [21](#), [22](#), [40](#), [41](#), [42](#)
- [30] Hanyu Zhao, Zhenhua Han, Zhi Yang, Quanlu Zhang, Fan Yang, Lidong Zhou, Mao Yang, Francis C. M. Lau, Yuqi Wang, Yifan Xiong, and Bin Wang. HiveD: Sharing a GPU cluster for deep learning with guarantees. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020. [8](#)